
# SDEV 2451

## Full Stack Development

## Frontend and Backend API Integration - 1

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

# Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

# Agenda

On the right is what we will cover today.

- Configure CORS in Django Settings
- Create the API Layer
- Create React Query Hooks
- Add QueryClientProvider to App
- Update Pages to Use Hooks
- Challenge — Error Feedback and Search Filter
- Key Takeaways

# Configure CORS in Django Settings

The browser blocks requests from one origin to another by default — `django-cors-headers` adds the response headers that tell the browser the request is allowed.

```
# settings.py
INSTALLED_APPS = [
    # ... django built-ins ...
    "corsheaders",      # register the package
    "rest_framework",
    "blog",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "corsheaders.middleware.CorsMiddleware",   # must be near the top
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    # ...
]

CORS_ALLOWED_ORIGINS = [
    "http://localhost:5173",   # Vite dev server
]
```

# Configure CORS in Django Settings (continued)

- `"corsheaders"` in `INSTALLED_APPS` registers the package so it can be used as middleware
- `CorsMiddleware` must sit **before** `SessionMiddleware` and `CommonMiddleware` — it needs to intercept preflight `OPTIONS` requests before any other middleware processes them
- `CORS_ALLOWED_ORIGINS` is an explicit allowlist; only origins listed here receive the `Access-Control-Allow-Origin` header
- Never use `CORS_ALLOW_ALL_ORIGINS = True` in production — it permits every domain to call your API

# Create the API Layer

Raw `fetch()` functions live in `src/api/` — plain JavaScript with no React dependencies, one file per resource.

```
// src/api/blog.js
const BASE_URL = 'http://localhost:8000/api/v1'

export async function fetchPosts() {
  const response = await fetch(`${BASE_URL}/posts/`)
  if (!response.ok) throw new Error('Failed to fetch posts')
  return response.json()
}

export async function createPost(data) {
  const response = await fetch(`${BASE_URL}/posts/`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  })
  if (!response.ok) throw new Error('Failed to create post')
  return response.json()
}
```

# Create the API Layer (continued)

- `BASE_URL` is defined once at the top — changing the server address requires editing only one line
- `if (!response.ok) throw new Error( ... )` — React Query catches this thrown error and exposes it via `isError` and `error` in the hook
- `return response.json()` returns the parsed data directly; React Query's `queryFn` caches whatever the function returns
- `'Content-Type': 'application/json'` is required on POST requests — DRF will reject the body without it
- No React imports — keeping HTTP logic in a plain JS file makes it easy to test and reuse

# Create React Query Hooks

Hooks in `src/hooks/` wrap the API functions and expose loading state, data, and errors — pages never call `fetch()` directly.

```
// src/hooks/usePosts.js
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { fetchPosts, createPost } from '../api/blog'

export function usePosts() {
  const { data: posts = [], isLoading, isError, error } = useQuery({
    queryKey: ['posts'],
    queryFn: fetchPosts,
  })
  return { posts, isLoading, isError, error }
}

export function useCreatePost() {
  const queryClient = useQueryClient()
  return useMutation({
    mutationFn: createPost,
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['posts'] }),
  })
}
```

# Create React Query Hooks (continued)

- `queryKey: ['posts']` uniquely identifies this query in the cache — two components calling `usePosts()` share one request and one cached result
- `data: posts = []` renames `data` and provides a default empty array — without it, `posts` is `undefined` on the first render and `.map()` crashes
- `useMutation` is for write operations; unlike `useQuery` it does not run automatically — the caller triggers it with `mutate(data)`
- `invalidateQueries({ queryKey: ['posts'] })` marks the cache stale after a successful POST so React Query re-fetches the list automatically
- Splitting into two exports ( `usePosts` read / `useCreatePost` write) keeps concerns separate — a read-only page doesn't import the mutation

# Add QueryClientProvider to App

React Query requires a shared `QueryClient` at the top of the component tree — every hook below it reads from the same cache.

```
// src/App.jsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom'

const queryClient = new QueryClient()   // outside the component

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <nav>{/* NavLinks */}</nav>
        <Routes>
          <Route path="/"          element={<PostsPage />} />
          <Route path="/posts/new" element={<CreatePostPage />} />
        </Routes>
      </BrowserRouter>
    </QueryClientProvider>
  )
}
```

# Add QueryClientProvider to App (continued)

- `new QueryClient()` is created **outside** the `App` function — if it were inside, a fresh empty cache would be created on every re-render
- `QueryClientProvider` must wrap every component that uses a React Query hook; placing it outside `BrowserRouter` ensures all pages and nested components have access
- The order of `QueryClientProvider` and `BrowserRouter` doesn't matter as long as both sit above the page components

# Update Pages to Use Hooks

Replace mock data imports with hooks — the component layer ( `PostList` , `PostForm` ) does not change at all.

```
// Before
import { POSTS } from '../mockData'

// After
import { usePosts } from '../hooks/usePosts'
import { useCreatePost } from '../hooks/usePosts'

function PostsPage() {
  const { posts, isLoading } = usePosts()
  return isLoading
    ? <span className="loading loading-spinner loading-md" />
    : <PostList posts={posts} />
}

function CreatePostPage() {
  const navigate = useNavigate()
  const { mutate: createPost, isPending } = useCreatePost()
  function handleSubmit(formData) {
    createPost(formData, { onSuccess: () => navigate('/posts') })
  }
```

# Update Pages to Use Hooks (continued)

- The only changes from the mock-data version are the import lines and the loading spinner — JSX structure, class names, and component usage are identical
- `isLoading` is `true` while the fetch is in flight; a DaisyUI spinner gives the user immediate feedback
- `mutate: createPost` renames the mutation trigger to describe what it does
- `onSuccess: () ⇒ navigate('/posts')` runs after the server responds with a success status — at this point the cache is already invalidated, so the list page will show the new item
- `isPending` is `true` while the POST is in flight — use it to show a spinner or disable the submit button

# Challenge — Error Feedback and Search Filter

1. Add error feedback to the pages
   - When `isError` is `true`, render a DaisyUI `alert alert-error` in place of the spinner or list
   - On the create page, show the error message below the form without navigating away
   - Test it by stopping the Django server and reloading the frontend
2. Add a search filter to the posts list
   - Add a controlled text input above `PostList` and store the search term in `useState`
   - Update `fetchPosts` in `src/api/blog.js` to accept an optional `search` string and append `?search=<term>` when present
   - Update `usePosts` to accept `search` as an argument, pass it to `fetchPosts`, and include it in the `queryKey`: `['posts', search]`
   - React Query re-fetches automatically whenever the key changes — no manual effect needed

# Key Takeaways

- **CORS** — browsers block cross-origin responses unless the server explicitly allows them; `django-cors-headers` adds the required headers
- `CorsMiddleware` **order** — must be placed before `SessionMiddleware` and `CommonMiddleware` to intercept preflight requests; wrong placement silently breaks all API calls
- `CORS_ALLOWED_ORIGINS` — explicit allowlist of trusted origins; `CORS_ALLOW_ALL_ORIGINS = True` is never safe in production
- `src/api/` **layer** — plain async functions that own the HTTP details; no React imports, easy to test in isolation
- `src/hooks/` **layer** — React Query hooks that expose `data`, `isLoading`, `isError`, and `error`; pages never call `fetch()` directly
- `useQuery` — caches data by `queryKey`; multiple components share one request; always provide a default value to avoid `undefined` on first render


# Example

Let’s go do an example together